CRYPTOGRAPHY BASED PASSWORD MANAGER

A Real-Time / Field-based Research Project (22CY284) report submitted to the Jawaharlal Nehru Technological University, Hyderabad

Submitted by

Name : S.JASHWANTH **Rollno : 24B81A6219**
Name : R.MURALI KRISHNA **Rollno : 24B81A6227**
Name : N.NIKSHAY KUMAR **Rollno : 24B81A6231**

Under the guidance of
Dr.R.Raja,Associate Professor

DEPARTMENT OF CSE (CS)

CVR COLLEGE OF ENGINEERING

(***An Autonomous Institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad***)
Vastunagar, Mangalpalli(V),Ibrahimpatnam(M),
Rangareddy (D), Telangana- 501 510

APRIL 2025

CVR COLLEGE OF ENGINEERING

(***An Autonomous Institution , NAAC Accredited and Affiliated to JNTUH, Hyderabad***)
Vastunagar,Mangalpalli(V),Ibrahimpatnam(M),
Rangareddy (D), Telangana- 501 510

DEPARTMENT OF CSE (CS)

CERTIFICATE

This is to certify that the Real-Time/ Field-based Research Project(22CY284) report entitled **"CRYPTOGRAPHY BASED PASSWORD MANAGER"** is a record of work carried out by **Name(s) : S.JASHWANTH , R.MURALI KRISHNA , N.NIKSHAY KUMAR** submitted to Department of **CSE (CS) ,** CVR College of Engineering, affiliated to Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026.

Project Guide	**Project Coordinator**	**Head of the Department**
Dr.R.Raja	**G. Sunitha Rekha**	**Dr. M. Sunitha**
Associate Professors	Sr. Assistant Professor	Professor & HOD
CSE(CS)	CSE(CS)	CSE(CS)

DECLARATION

We hereby declare that the Real-Time/ Field-based Research Project (22CY284) report entitled CRYPTOGHAPHY BASED PASSWORD MANAGER is an original work done and submitted to **CSE (CS)** Department, CVR College of Engineering, affiliated to Jawaharlal Nehru Technological University, Hyderabad and it is a record of bonafide project work carried out by us under the guidance of **Dr.RAJA**, **Associate Professor , CSE(CS).**

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

Signature of the Student

NAME : S.JASHWANTH

Roll No : 24B81A6219

Signature of the Student

NAME : R.MURALI KRISHNA

Roll No : 24B81A6227

Signature of the Student

Name : N.NIKSHAY KUMAR

Roll No : 24B81A6231

ACKNOWLEDGEMENT

We wish to express a deep sense of gratitude and heartfelt thanks to **Dr. Rama Mohan Reddy**, Principal, and the Management for providing excellent lab facilities and resources.

We thank our Vice-Principal **Prof. L. C. Siva Reddy** for providing excellent computing facilities and a disciplined atmosphere for doing our work.

We thank **Mr.B.MAHENDER REDDY**, Project Coordinator and **Dr.M.Radha and Dr.V.Deepika ,** Project Review Committee members for their valuable guidance and support which helped us to complete the project work successfully.

We respect and thank our internal guide Dr.**R.RAJA, Associate Profressors , CSE(CS)** , for giving us all the support and guidance, which made us complete the project duly.

We would like to express heartfelt thanks to **Dr. M. Sunitha,** Professor & Head of the Department, for providing us with an opportunity to do this project and extending support and guidance.

We would also like to express our sincere gratitude to **Dr. H. N. Lakshmi**, Assoc. Dean, ET, for her constant encouragement, valuable suggestions, and support throughout the course of this project.

Finally, we thank all those who directly or indirectly contributed to the successful completion of this project.

TABLE OF CONTENTS

Chapter No.		Contents	Page No.
		Certificate	i
		Declaration	ii
		Acknowledgement	iii
		List of Figures	iv
		List of Tables (if any)	v
		Abstract	vi
1		**Introduction**	
	1.1	Problem Statement	7
	1.2	Project Objectives	7
	1.3	Software & Hardware specifications	7
	1.3.1	Software requirements	7
	1.3.2	Hardware requirements	7
2		**Design Methodology**	
	2.1	System Architecture	7
	2.2	Data flow Diagram or Flowchart	8
	2.3	Technology Description	8
3		**Implementation & Testing**	
	3.1	Code snippets	8
	3.2	Test cases	8
4		**Conclusion**	**9**
		Bibliography	**9**
		Appendix: (Source code)	**10-16**

VaultPy Documentation

1 Introduction

Password management is an ongoing challenge, and remembering multiple strong passwords is often difficult. We wanted to build an application that runs locally, keeps data secure, and is easy to use without relying on a cloud service. Therefore, we created VaultPy, a Python-based password manager that stores everything securely on the local machine using reliable encryption instead of plain text.

1.1Problem Statement

Many people tend to reuse the same weak passwords across different accounts because remembering unique, strong passwords is too hard. While cloud-based password managers exist, some users prefer not to store their private data on remote servers. There is a clear need for a fast, straightforward, and local alternative that guarantees privacy while offering strong encryption to protect stored credentials.

1.2ProjectObjectives

- To build a simple, easy-to-use desktop application for managing passwords.
- To ensure maximum security by encrypting the data locally using AES-256-GCM.
- To protect the master password against cracking attempts using Argon2id hashing.
- To present a clean, modern user interface that behaves like a web app but runs natively on the desktop.

1.3 Software & Hardware specifications

1.3.1 Software requirements

- **Operating System:** Windows 10 or higher
- **Language:** Python 3.x
- **Frameworks/Libraries:** Flask (for backend), PyWebView (for desktop UI), Cryptography (AES-256), Argon2-cffi (hashing)
- **Frontend:** HTML, CSS, JavaScript (Vanilla)

1.3.2 Hardware requirements

- **Processor:** Dual-core CPU or better (Intel i3/Ryzen 3 equivalent)
- **RAM:** Minimum 2GB (4GB recommended)
- **Storage:** 100 MB of free storage space for the application and the vault

2 Design Methodology

2.1 System Architecture

VaultPy functions asa local web application wrapper. We use a Flask backend that handles all data operations and cryptography. PyWebView serves as the frontend client, displaying the user interface naturally on the desktop. The backend communicates with a local encrypted file (vault.enc) to read and update user entries. Importantly, no data ever leaves the local machine.

2.2 Data flow Diagram or Flowchart

Forthe data flow:

1. The user launches the application and enters the Master Password.
2. The system hashes the password using Argon2id to derive a secure 256-bit encryption key.
3. This key is used to decrypt the vault.enc file using AES-256-GCM.
4. If successful, the Flask server securely passes the decrypted entries to the PyWebView frontend.
5. When a user adds a new entry, the frontend sends it to Flask, which updates the list in memory, re-encrypts the data, and saves the secure file on disk.

2.3 Technology Description

- **Python:**The coreprogramming language used to build the application logic.
- **Flask:** Used to create the local web server that bridges the UI and the logic.
- **PyWebView:** Displays the Flask application in a standalone desktop window without a standard browser toolbar, giving it the feel of a native application.
- **Cryptography (AES-256-GCM):** An authenticated encryption standard used to secure the vault configuration file.
- **Argon2id:** A robust key derivation function to process the master password securely, protecting it against brute-force attacks.

3 Implementation & Testing

3.1 Code snippets

Derivingthekeyusing Argon2id:

```
key = hash_secret_raw(

    secret=master_password.encode("utf-8"),
    salt=salt,
    time_cost=3,
    memory_cost=65536,
    parallelism=4,
    hash_len=32,
```

```
    type=Type.ID
)
```

Encrypting data using AES-GCM:

```
aesgcm = AESGCM(key)
nonce = os.urandom(12)
ciphertext = aesgcm.encrypt(nonce, plaintext.encode("utf-8"), None)
```

3.2 Testcases

- Test Case 1: Entering the correct master password.
  Expected: Vault unlocks and shows dashboard.
  Result: Pass.
- Test Case 2: Entering an incorrect master password.
  Expected: Application rejects access and shows an error message.
  Result: Pass.
- Test Case 3: Adding a new password entry.
  Expected: Entry is displayed in the list and saved encrypted on the local disk.
  Result: Pass.
- Test Case 4: Copying a password.
  Expected: Password copies to the clipboard and clears out automatically after 30 seconds for security purposes.
  Result: Pass.

4 Conclusion

In conclusion, VaultPy successfully meets our goal of building a lightweight, secure, and fully local password manager. Working on this project allowed us to practically learn about cybersecurity and cryptography, specifically implementing AES-256-GCM and Argon2id. It is a functional application that addresses the problem of managing passwords safely without needing to rely on third-party cloud tools.

Bibliography

- Python Official Documentation: https://docs.python.org/3/
- Cryptography PyPI Documentation: https://cryptography.io/en/latest/
- Flask Web Development guide
- Argon2 documentation andsecurity guidelines

Appendix: (Source code)

Thesearethemainfilesrequired to set up the VaultPy application logic and backend.

app.py

```
"""
app.py — Flask application entry point

---------------------------------------
Runs locally on 127.0.0.1 inside a native pywebview window.
Run with: python app.py
"""

import os
import sys
import threading
import time
import webview
from flask import (Flask, render_template, request,
                   redirect, url_for, session, jsonify)
from vault_manager import VaultManager


# ------------------------------------------------------------------ #
#  Resource path helper (needed when running as PyInstaller .exe)     #
# ------------------------------------------------------------------ #

def resource_path(rel):
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, rel)
    return os.path.join(os.path.abspath("."), rel)


# ------------------------------------------------------------------ #
#  Flask app setup                                                     #
# ------------------------------------------------------------------ #

app = Flask(__name__,
            template_folder=resource_path("templates"),
            static_folder=resource_path("static"))

app.secret_key = os.urandom(32)

vault = VaultManager()


# ------------------------------------------------------------------ #
#  Helper                                                              #
# ------------------------------------------------------------------ #

def require_unlocked():
    if not vault.is_unlocked:
        return redirect(url_for("unlock"))
    return None


# ------------------------------------------------------------------ #
#  Routes                                                              #
# ------------------------------------------------------------------ #
```

```
@app.route("/")
def index():
    if vault.is_unlocked:
        return redirect(url_for("dashboard"))
    return redirect(url_for("unlock"))


@app.route("/unlock", methods=["GET", "POST"])
def unlock():
    if request.method == "POST":
        password = request.form.get("master_password", "")
        if not vault.vault_exists():
            vault.create_vault(password)
            session["unlocked"] = True
            return redirect(url_for("dashboard"))
        success = vault.unlock(password)
        if success:
            session["unlocked"] = True
            return redirect(url_for("dashboard"))
        else:
            return render_template("unlock.html",
                                   error="Wrongmasterpassword. Try again.",
                                   is_new=False)
    is_new = not vault.vault_exists()
    return render_template("unlock.html", is_new=is_new)


@app.route("/dashboard")
def dashboard():
    guard = require_unlocked()
    if guard:
        return guard
    entries = vault.get_entries()
    returnrender_template("dashboard.html",entries=entries)


@app.route("/add", methods=["GET", "POST"])
def add_entry():
    guard = require_unlocked()
    if guard:
        return guard
    if request.method == "POST":
        vault.add_entry(
            title    = request.form.get("title", ""),
            username = request.form.get("username", ""),
            password = request.form.get("password", ""),
            url      = request.form.get("url", ""),
            notes    = request.form.get("notes", "")
        )
        return redirect(url_for("dashboard"))
    return render_template("add_entry.html")


@app.route("/delete/<entry_id>", methods=["POST"])
def delete_entry(entry_id):
    guard = require_unlocked()
    if guard:
        return guard
    vault.delete_entry(entry_id)
    return redirect(url_for("dashboard"))
```

```
@app.route("/lock")
def lock():
    vault.lock()
    session.clear()
    return redirect(url_for("unlock"))


@app.route("/api/password/<entry_id>")
def get_password(entry_id):
    if not vault.is_unlocked:
        return jsonify({"error": "locked"}), 403
    entry = vault.get_entry(entry_id)
    if not entry:
         return jsonify({"error": "not found"}), 404
     return jsonify({"password": entry["password"]})


# ---------------------------------------------------------------- #
#  Launch                                                           #
# ---------------------------------------------------------------- #

def start_flask():
    """Run Flask in a background thread."""
    app.run(host="127.0.0.1", port=5000, debug=False, use_reloader=False)


if __name__ == "__main__":
    # Start Flask in background thread
    flask_thread = threading.Thread(target=start_flask)
    flask_thread.daemon = True
    flask_thread.start()

    # Wait briefly for Flask to be ready
    time.sleep(1)

    # Open native desktop window — no browser, no address bar
    window = webview.create_window(
        title     = "VaultPy — Password Manager",
        url       = "http://127.0.0.1:5000",
        width     = 1024,
        height    = 700,
        min_size = (800, 600),
        resizable = True
    )
    webview.start()
```

crypto.py

```
"""
crypto.py — Core cryptography module for VaultPy
-------------------------------------------------
Handles:
  - Argon2id key derivation from master password
  - AES-256-GCM authenticated encryption / decryption
  - Secure salt and nonce generation
"""

import os
```

```
import json
import base64
from argon2.low_level import hash_secret_raw, Type
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


# --- Argon2id Parameters (tuned for security vs speed) ---
ARGON2_TIME_COST  =   3   =   # iterations
ARGON2_MEMORY_COST  65536  =  # 64 MB
ARGON2_PARALLELISM  4 = 32 =
ARGON2_HASH_LEN     16         # 256-bit key output
ARGON2_SALT_LEN                # 128-bit salt

# --- AES-GCM Parameters ---
NONCE_LEN = 12   # 96-bit nonce (recommended for GCM)


def generate_salt() -> bytes:
    """Generate a cryptographically secure random salt."""
    return os.urandom(ARGON2_SALT_LEN)


def derive_key(master_password: str, salt: bytes) -> bytes:
    """
    Derivea256-bitencryptionkey     fromthemasterpasswordusing Argon2id.

    Args:
        master_password: The user's master password (string)
        salt:Randomsalt   bytes(generatedonce,stored    invault file)

    Returns:
        32-bytederivedkey(neverstored,onlykept       in memory)
    """
    key = hash_secret_raw(
        secret=master_password.encode("utf-8"),
        salt=salt,
        time_cost=ARGON2_TIME_COST,
        memory_cost=ARGON2_MEMORY_COST,
        parallelism=ARGON2_PARALLELISM,
        hash_len=ARGON2_HASH_LEN,
        type=Type.ID   #Argon2idvariant
    )
    return key


def encrypt(plaintext: str, key: bytes) -> dict:
    """
    Encrypt plaintextusingAES-256-GCMauthenticatedencryption.

    Args:
        plaintext: String data to encrypt (your vault JSON)
        key: 32-byte derived key from derive_key()

    Returns:
        Dictwithnonceandciphertext     (bothbase64-encoded  for storage)
    """
    aesgcm = AESGCM(key)
    nonce=  os.urandom(NONCE_LEN)                          #fresh nonce every time
    ciphertext =aesgcm.encrypt(nonce,  plaintext.encode("utf-8"), None)
```

```
    return {
        "nonce":      base64.b64encode(nonce).decode(),
        "ciphertext":base64.b64encode(ciphertext).decode()
        #Note:AESGCM   appendsthe16-byteauth tag to ciphertext automatically
    }


defdecrypt(encrypted:  dict, key:bytes)  ->str:
    """
    Decrypt andverify  AES-256-GCMciphertext.

    Args:
        encrypted:Dictwith   'nonce'and'ciphertext' (base64 strings)
        key:32-bytederived   key fromderive_key()

    Returns:
        Decrypted plaintext string

    Raises:
        cryptography.exceptions.InvalidTag:If decryption fails (wrong key or tampered data)
    """
    aesgcm = AESGCM(key)
    nonce      =base64.b64decode(encrypted["nonce"])
    ciphertext =base64.b64decode(encrypted["ciphertext"])
    plaintext  =aesgcm.decrypt(nonce,  ciphertext, None)
    return plaintext.decode("utf-8")


def wipe_key(key: bytearray) -> None:
    """
    Overwrite key material in memory before discarding.
    Call thiswhenlockingthevaultoronapp shutdown.
    """
    if isinstance(key, bytearray):
        for i in range(len(key)):
            key[i] = 0
```

vault_manager.py

```
"""
vault_manager.py — Vault file read/write logic
----------------------------------------------
The vault file (~/.vaultpy/vault.enc) is a JSON file structured as:

{
  "kdf": "argon2id",
  "kdf_params": { "t": 3, "m": 65536, "p": 4 },
  "salt": "<base64>",
  "nonce": "<base64>",
  "ciphertext": "<base64>"
}

The encrypted payload is a JSON array of password entries:
[
  {
    "id": "uuid",
    "title": "GitHub",
    "username": "alice@email.com",
    "password": "s3cr3t",
```

```
    "url": "https://github.com",
    "notes": ""
  },
  ...
]
"""

import os
import json
import base64
import uuid
from pathlib import Path
from cryptography.exceptions import InvalidTag
from crypto import derive_key, encrypt, decrypt, generate_salt


# Default vault location: ~/.vaultpy/vault.enc
VAULT_DIR = Path.home() / ".vaultpy"
VAULT_FILE = VAULT_DIR / "vault.enc"


class VaultManager:
    """
    Manages the encrypted vault file and in-memory vault state.

    Usage:
        vm = VaultManager()

        # First time — create vault
        vm.create_vault("my_master_password")

        # Later — unlock existing vault
        vm.unlock("my_master_password")
        entries = vm.get_entries()

        # Add a new entry
        vm.add_entry("GitHub","alice@email.com",  "s3cr3t", "https://github.com")

        # Save and lock
        vm.save()
        vm.lock()
    """

    def __init__(self):
        self._key:bytes|None=None              # Derivedkey,  lives inmemory  only
        self._entries: list[dict]= []           #Decryptedvaultentries
        self._salt: bytes |None =None           #Storedinvaultfile
        self._unlocked: bool = False

    # ---------------------------------------------------------------- #
    # Vaultlifecycle                                                   #
    # ---------------------------------------------------------------- #

    def vault_exists(self) -> bool:
        return VAULT_FILE.exists()

    def create_vault(self, master_password: str) -> None:
        """Create a brand new empty vault with a fresh salt."""
        VAULT_DIR.mkdir(parents=True, exist_ok=True)
```

```
        self._salt    =generate_salt()
        self._key     =derive_key(master_password,self._salt)
        self._entries = []
        self._unlocked = True

        self.save()

        # Lock down file permissions (owner read/write only)
        os.chmod(VAULT_FILE, 0o600)

    def unlock(self, master_password: str) -> bool:
        """
        Derive key from master password and decrypt vault.
        Returns True on success, False on wrong password.
        """
        if not self.vault_exists():
            raiseFileNotFoundError("Novaultfound.    Create one first.")

        with open(VAULT_FILE, "r") as f:
            vault_data = json.load(f)

        self._salt = base64.b64decode(vault_data["salt"])
        self._key = derive_key(master_password, self._salt)

        try:
            plaintext = decrypt(
                {"nonce": vault_data["nonce"], "ciphertext":vault_data["ciphertext"]},
                self._key
            )
            self._entries = json.loads(plaintext)
            self._unlocked = True
            return True

        except InvalidTag:
            # Wrong password or tampered file
            self._key     =None
            self._entries = []
            return False

    def lock(self) -> None:
        """Wipe key and entries from memory."""
        self._key      =None
        self._entries =[]
        self._unlocked = False

    def save(self) -> None:
        """Encrypt and write vault to disk."""
        if not self._unlocked or self._key is None:
            raise PermissionError("Vault is locked.")

        plaintext   =json.dumps(self._entries)
        encrypted   =encrypt(plaintext,self._key)

        vault_data = {
            "kdf": "argon2id",
            "kdf_params": {"t": 3, "m": 65536, "p": 4},
            "salt":       base64.b64encode(self._salt).decode(),
            "nonce":      encrypted["nonce"],
            "ciphertext": encrypted["ciphertext"]
        }
```

```
        VAULT_DIR.mkdir(parents=True, exist_ok=True)
        with open(VAULT_FILE, "w") as f:
            json.dump(vault_data, f, indent=2)

    # ---------------------------------------------------------------- #
    # Entry CRUD                                                       #
    # ---------------------------------------------------------------- #

    def get_entries(self) -> list[dict]:
        return self._entries.copy()

    def add_entry(self, title: str, username: str, password: str,
                  url: str = "", notes: str = "") -> dict:
        entry = {
        "id":           str(uuid.uuid4()),
        "title":        title,
            "username": username,
            "password": password,
            "url":      url,
            "notes":    notes
        }
        self._entries.append(entry)
        self.save()
        return entry

    def update_entry(self, entry_id: str, **fields) -> bool:
        for entry in self._entries:
            if entry["id"] == entry_id:
                entry.update(fields)
                self.save()
                return True
        return False

    def delete_entry(self, entry_id: str) -> bool:
        before = len(self._entries)
        self._entries = [e for e in self._entries if e["id"] != entry_id]
        if len(self._entries) < before:
            self.save()
            return True
        return False

    def get_entry(self, entry_id: str) -> dict | None:
        return next((e for e in self._entries if e["id"] == entry_id), None)

    @property
    def is_unlocked(self) -> bool:
        return self._unlocked
```

requirements.txt

```
flask>=3.0.0
argon2-cffi>=23.1.0
cryptography>=42.0.0
pywebview>=5.0.0
python-docx>=1.1.0
docx2pdf>=0.1.8
```